

Verification and Reasoning-focused post-training

Oct 31, 2025

LLMs with Verification

Training Verifiers to Solve Math Word Problems

Karl Cobbe*	Vineet Kosaraju*	Mohammad Bavarian	Mark Chen
Heewoo Jun	Lukasz Kaiser	Matthias Plappert	Jerry Tworek
Jacob Hilton	Reiichiro Nakano	Christopher Hesse	John Schulman

OpenAI

Abstract

State-of-the-art language models can match human performance on many tasks, but they still struggle to robustly perform multi-step mathematical reasoning. To diagnose the failures of current models and support research, we introduce GSM8K, a dataset of 8.5K high quality linguistically diverse grade school math word problems. We find that even the largest transformer models fail to achieve high test performance, despite the conceptual simplicity of this problem distribution. To increase performance, we propose training verifiers to judge the correctness of model completions. At test time, we generate many candidate solutions and select the one ranked highest by the verifier. We demonstrate that verification significantly improves performance on GSM8K, and we provide strong empirical evidence that verification scales more effectively with increased data than a finetuning baseline.

1 Introduction

In recent years, large language models have demonstrated impressive skills across many diverse tasks (Wang et al., 2019; Brown et al., 2020). Kaplan et al. (2020) describe the consistent benefits of increasing model size, characterizing scaling trends that hold across many orders of magnitude. However, even the largest models falter when required to perform multi-step mathematical reasoning (Hendrycks et al., 2021). Model samples frequently contain catastrophic mistakes, even after the model has been appropriately finetuned. Mathematical reasoning thus reveals a critical weakness in modern language models.

One significant challenge in mathematical reasoning is the high sensitivity to individual mistakes (Shen et al., 2021a). When generating a solution, autoregressive models have no mechanism to correct their own errors. Solutions that veer off-course quickly become unrecoverable. If we rely purely on generative methods and extrapolate from current trends, we will require an exorbitant

*Equal contribution. Correspondence to: Karl Cobbe <karl@openai.com>, Vineet Kosaraju <vineet@openai.com>

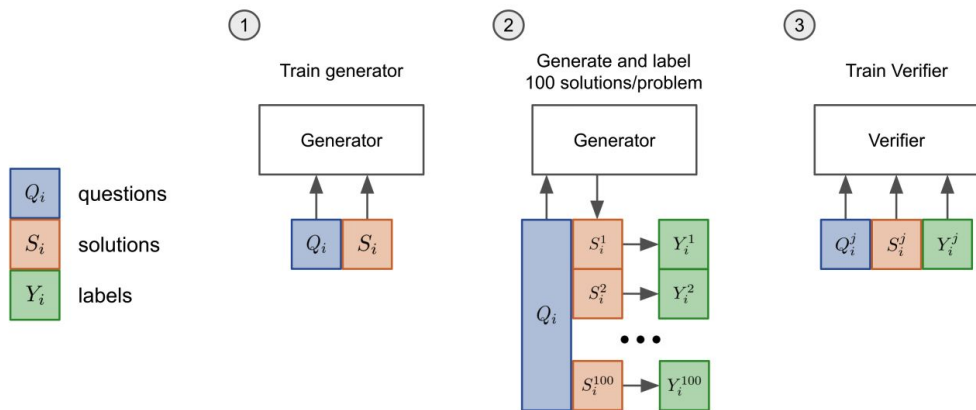
Finetuning vs verification

Finetuning: Train a generator (GPT-3) on CoT data and at inference time use the generator to generate response with temperature $T=0$.

Finetuning vs verification

Verification:

1. Train a generator
2. Sample 100 completions with labels (correct vs incorrect)
3. Train a verifier on this dataset.



“solution” is CoT + answer.

Figure 4: A diagram of the verification training pipeline.

Finetuning vs verification

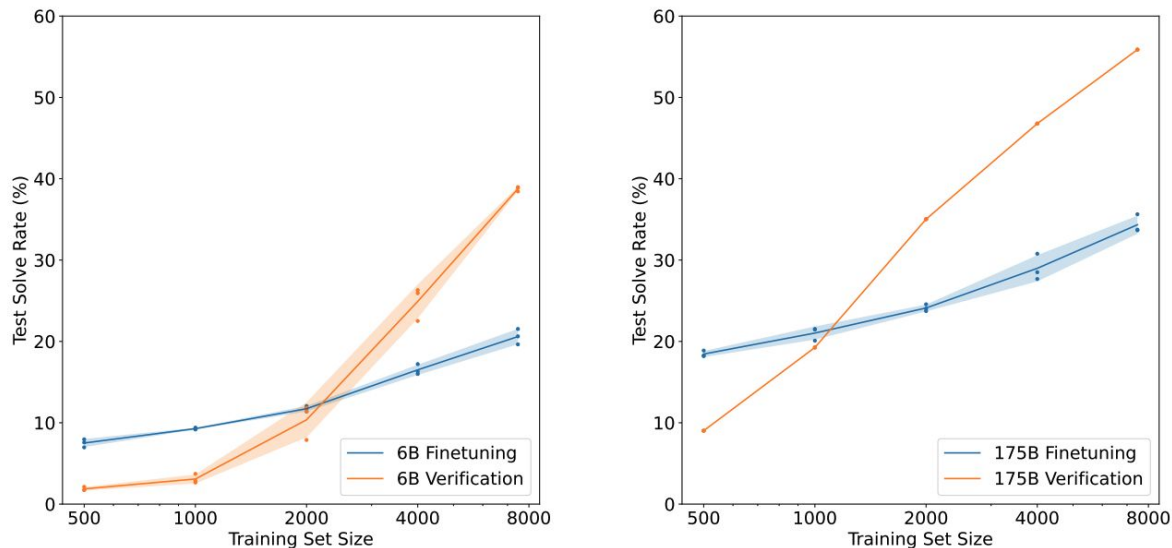


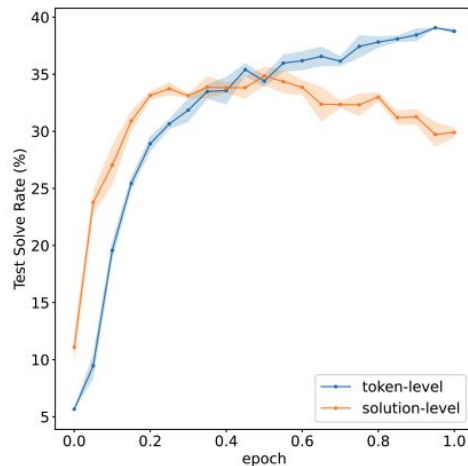
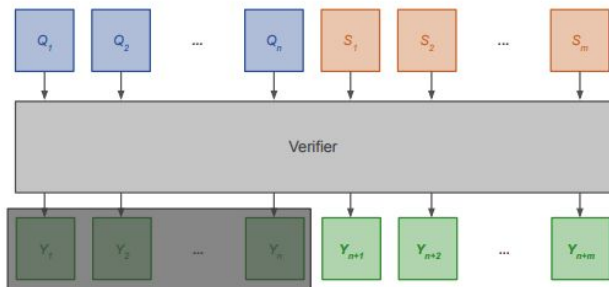
Figure 5: A comparison between finetuning and verification using 6B and 175B model sizes. Verification considers 100 solutions per problem. Mean and standard deviation is shown across 3 runs, except for 175B verification which shows only a single run.

Training verifier – two types of loss

solution-level loss: Train verifier to make a single scalar prediction conditioned on the entire generated solution.

token-level loss: Train verifier to make a scalar prediction after each token in the solution.

Verifier Objective



(a) Comparison between a verifier trained to predict correctness after every token (token-level) and one trained to predict correctness after only the final token (solution-level)

Let’s Verify Step by Step

Hunter Lightman* Vineet Kosaraju* Yura Burda* Harri Edwards
Bowen Baker Teddy Lee Jan Leike John Schulman Ilya Sutskever
Karl Cobbe*
OpenAI

Abstract

In recent years, large language models have greatly improved in their ability to perform complex multi-step reasoning. However, even state-of-the-art models still regularly produce logical mistakes. To train more reliable models, we can turn either to outcome supervision, which provides feedback for a final result, or process supervision, which provides feedback for each intermediate reasoning step. Given the importance of training reliable models, and given the high cost of human feedback, it is important to carefully compare the both methods. Recent work has already begun this comparison, but many questions still remain. We conduct our own investigation, finding that process supervision significantly outperforms outcome supervision for training models to solve problems from the challenging MATH dataset. Our process-supervised model solves 78% of problems from a representative subset of the MATH test set. Additionally, we show that active learning significantly improves the efficacy of process supervision. To support related research, we also release PRM800K, the complete dataset of 800,000 step-level human feedback labels used to train our best reward model.

1 Introduction

Large language models are capable of solving tasks that require complex multi-step reasoning by generating solutions in a step-by-step chain-of-thought format (Nye et al., 2021; Wei et al., 2022; Kojima et al., 2022). However, even state-of-the-art models are prone to producing falsehoods — they exhibit a tendency to invent facts in moments of uncertainty (Bubeck et al., 2023). These *hallucinations* (Maynez et al., 2020) are particularly problematic in domains that require multi-step reasoning, since a single logical error is enough to derail a much larger solution. Detecting and mitigating hallucinations is essential to improve reasoning capabilities.

*Primary authors. Correspondence to: Karl Cobbe <karl@openai.com>

ORM and PRM

Outcome-supervised reward model (ORM): trained using only the final result of the model's CoT.

Process-supervised reward model (PRM): receive feedback for each step in the CoT.

PRM

Let

$$x^8 + 3x^4 - 4 = p_1(x)p_2(x) \cdots p_k(x),$$

where each non-constant polynomial $p_i(x)$ is monic with integer coefficients, and cannot be factored further over the integers. Compute $p_1(1) + p_2(1) + \cdots + p_k(1)$.

I notice that the given polynomial has even degree and only even powers of x , so I can try to make a substitution to simplify it.

Let $y = x^4$, then the polynomial becomes $y^2 + 3y - 4$, which is a quadratic equation.

I can factor this quadratic equation as $(y + 4)(y - 1)$, so the original polynomial is $(x^4 + 4)(x^4 - 1)$.

Now I need to factor each of these factors further, if possible, over the integers.

For the first factor, $x^4 + 4$, I recall the Sophie Germain identity, which says that $a^4 + 4b^4 = (a^2 + 2b^2 + 2ab)(a^2 + 2b^2 - 2ab)$.

Applying this identity with $a = x$ and $b = 1$, I get $x^4 + 4 = (x^2 + 2x + 2)(x^2 - 2x + 2)$.

For the second factor, $x^4 - 1$, I can use the difference of squares formula, which says that $a^2 - b^2 = (a + b)(a - b)$.

Applying this formula with $a = x^2$ and $b = 1$, I get $x^4 - 1 = (x^2 + 1)(x^2 - 1)$.

Then I can apply the difference of squares formula again with $a = x$ and $b = 1$, to get $x^2 - 1 = (x + 1)(x - 1)$.

So the original polynomial is $(x^4 + 4)(x^4 - 1) = (x^2 + 2x + 2)(x^2 - 2x + 2)(x^2 + 1)(x + 1)(x - 1)$.

These are all the irreducible factors over the integers, so $p_1(x) = x^2 + 2x + 2$, $p_2(x) = x^2 - 2x + 2$, $p_3(x) = x^2 + 1$, $p_4(x) = x + 1$, and $p_5(x) = x - 1$.

To find the sum of their values at $x = 1$, I just need to plug in $x = 1$ into each factor and add them up.

I get $p_1(1) = 1^2 + 2 \cdot 1 + 2 = 5$, $p_2(1) = 1^2 - 2 \cdot 1 + 2 = 1$, $p_3(1) = 1^2 + 1 = 2$, $p_4(1) = 1 + 1 = 2$, and $p_5(1) = 1 - 1 = 0$.

So the final answer is $5 + 1 + 2 + 2 + 0 = 10$.

Answer: 10

Let

$$x^8 + 3x^4 - 4 = p_1(x)p_2(x) \cdots p_k(x),$$

where each non-constant polynomial $p_i(x)$ is monic with integer coefficients, and cannot be factored further over the integers. Compute $p_1(1) + p_2(1) + \cdots + p_k(1)$.

This problem is about factoring a polynomial over the integers, which can be a tricky task.

One possible strategy is to look for rational roots of the polynomial, using the rational root theorem.

However, that would involve checking a lot of possible factors of -4 , and there is no guarantee that the polynomial has any rational roots at all.

Another possible strategy is to use some algebraic tricks to simplify the polynomial or reduce its degree.

I notice that the polynomial has only even powers of x , so I can try to substitute $y = x^2$ and see what happens.

Then the polynomial becomes $y^4 + 3y^2 - 4$, which is a quadratic equation in y^2 .

I can try to factor this quadratic equation by looking for two numbers that multiply to -4 and add to 3 .

Those numbers are 4 and -1 , so I can write $y^4 + 3y^2 - 4 = (y^2 + 4)(y^2 - 1)$.

Now I can substitute back $x^2 = y$ and get $x^8 + 3x^4 - 4 = (x^4 + 4)(x^4 - 1)$.

This looks promising, because I have reduced the degree of the factors from 8 to 4 .

I can try to factor each of these factors further, using the difference of squares formula.

I get $x^8 + 3x^4 - 4 = (x^2 + 2)(x^2 - 2)(x^2 + 1)(x^2 - 1)$.

I can apply the difference of squares formula again to the last factor and get $x^8 + 3x^4 - 4 = (x^2 + 2)(x^2 - 2)(x^2 + 1)(x + 1)(x - 1)$.

Now I have factored the polynomial completely into monic linear and quadratic factors with integer coefficients.

These are the $p_i(x)$'s that the problem is asking for.

To find the sum of their values at $x = 1$, I just need to plug in $x = 1$ into each factor and add them up.

I get $p_1(1) + p_2(1) + \cdots + p_k(1) = (1^2 + 2)(1^2 - 2)(1^2 + 1)(1 + 1)(1 - 1)$.

Simplifying, I get $p_1(1) + p_2(1) + \cdots + p_k(1) = (3)(-1)(2)(2)(0)$.

Multiplying, I get $p_1(1) + p_2(1) + \cdots + p_k(1) = 0$.

Answer: 0

Figure 2: Two solutions to the same problem, graded by the PRM. The solution on the left is correct while the solution on the right is incorrect. A green background indicates a high PRM score, and a red background indicates a low score. The PRM correctly identifies the mistake in the incorrect solution.

Data collection

Human data-labelers are presented with step-by-step solution to MATH problems and task is to assign solution in {positive, negative, neutral}.

Surface *convincing wrong-answer* solution

- “convincing”: solutions that are rated highly by current best PRM.
- “wrong”: solutions that reach an incorrect final answer.

Data collection

The denominator of a fraction is 7 less than 3 times the numerator. If the fraction is equivalent to $\frac{2}{5}$, what is the numerator of the fraction? (Answer:)

   Let's call the numerator x .

   So the denominator is $3x-7$.

   We know that $x/(3x-7) = 2/5$.

   So $5x = 2(3x-7)$.

   $5x = 6x - 14$.

   So $x = 7$.

Figure 1: A screenshot of the interface used to collect feedback for each step in a solution.

Training PRM

- Train PRM to predict the correctness of each step after the last token in each step.
 - “This prediction takes the form of a single token, and we maximize the log-likelihood of these target tokens during training. The PRM can therefore be trained in a standard language model pipeline without any special accommodations.”
- Supervise only up to the first incorrect step.
 - Comparison between ORM
 - Cost-effective

Scoring

Define PRM score for a solution to be the probability that every step is correct under PRM as product of correctness probabilities.

Scoring

Ablation on scoring:

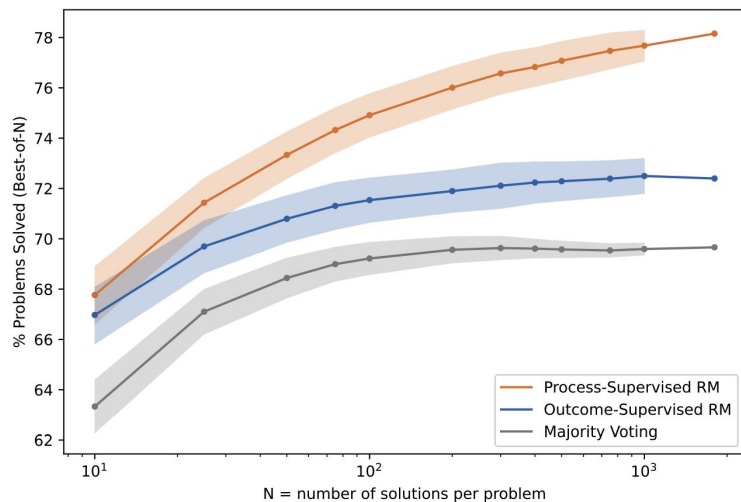
- Neutral label positive or negative
- Solution-level score as minimum score or product scores

	product	minimum
neutral = positive	78.2%	77.6%
neutral = negative	77.4%	77.8%

Table 4: Best-of-1860 test performance using the PRM with four different scoring strategies.

ORM vs PRM

	ORM	PRM	Majority Voting
% Solved (Best-of-1860)	72.4	78.2	69.6



For ORM, train on 100 uniform samples per problem from the generator (an order of magnitude larger than PRM).

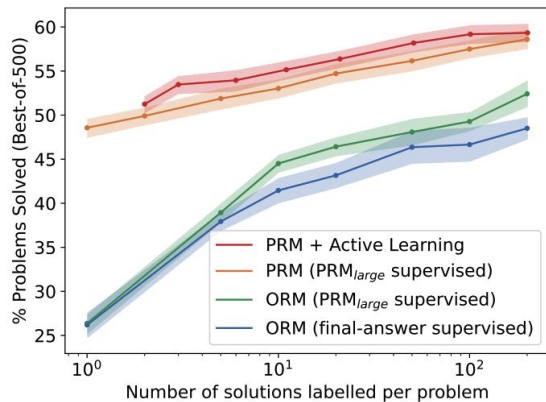
Figure 3: A comparison of outcome-supervised and process-supervised reward models, evaluated by their ability to search over many test solutions. Majority voting is shown as a strong baseline. For $N \leq 1000$, we visualize the variance across many subsamples of the 1860 solutions we generated in total per problem.

ORM vs PRM (a smaller setting)

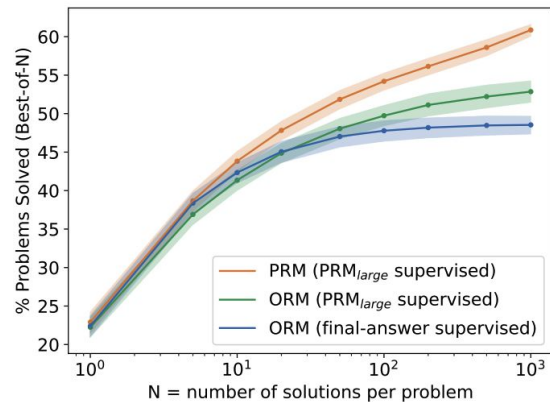
Build a small dataset with three supervision data. Sample 1-200 solutions per problem and use

1. Process supervision from PRM_{large}.
2. Outcome supervision from PRM_{large}.
3. Outcome supervision from final-answer checking.

ORM vs PRM (a smaller setting)



(a) Four series of reward models trained using different data collection strategies, compared across training sets of varying sizes.



(b) Three reward models trained on 200 samples/problem using different forms of supervision, compared across many test-time compute budgets.

Figure 4: A comparison of different forms of outcome and process supervision. Mean and standard deviation is shown across three seeds.

Generative Verifiers: Reward Modeling as Next-Token Prediction

Lunjun Zhang^{1,2}, Arian Hosseini^{1,3*}, Hritik Bansal^{1,4*}, Mehran Kazemi¹, Aviral Kumar^{1,5} and Rishabh Agarwal¹

*Core Contribution, ¹Google DeepMind, ²University of Toronto, ³Mila, ⁴UCLA, ⁵Carnegie Mellon University

Verifiers or reward models are often used to enhance the reasoning performance of large language models (LLMs). A common approach is the Best-of-N method, where N candidate solutions generated by the LLM are ranked by a verifier, and the best one is selected. While LLM-based verifiers are typically trained as discriminative classifiers to score solutions, they do *not* utilize the text generation capabilities of pretrained LLMs. To overcome this limitation, we instead propose training verifiers using the ubiquitous next-token prediction objective, jointly on verification and solution generation. Compared to standard verifiers, such generative verifiers (GenRM) can benefit from several advantages of LLMs: they integrate seamlessly with instruction tuning, enable chain-of-thought reasoning, and can utilize additional test-time compute via majority voting for better verification. We demonstrate that GenRM outperforms discriminative, DPO verifiers, and LLM-as-a-Judge, resulting in large performance gains with Best-of-N, namely 5% $\rightarrow$ 45.3% on algorithmic tasks and 73% $\rightarrow$ 93.4% on GSM8K. In easy-to-hard generalization settings, we observe improvements of 28% $\rightarrow$ 44.6% on MATH, and 37.9% $\rightarrow$ 53.5% on MMLU abstract algebra. Furthermore, we find that training GenRM with synthetic verification rationales is sufficient to pick out subtle errors on math problems. Finally, we demonstrate that GenRM scales favorably with model size and test-time compute.

1. Introduction

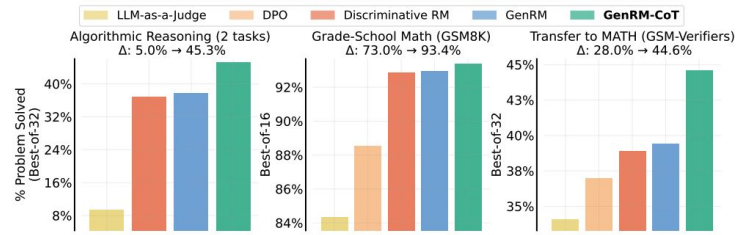


Figure 1 | **Generative Verifiers outperform standard verification approaches** in terms of Best-of-N on reasoning tasks, with a fixed generator. Here, Δ represents the improvement in number of problems solved with Best-of-N using GenRM-CoT. GenRM-CoT leverages the generation capabilities of LLMs, enabling a finetuned verifier to utilize chain-of-thought verification to detect subtle reasoning errors. For algorithmic tasks, we report average performance using Gemma-2B on Last Letter Concat (Wei et al., 2022) and BBH Word Sorting (Suzgun et al., 2022). For math reasoning, we train Gemma2-9B verifiers on GSM8K and evaluate their performance on GSM8K test (middle) and *easy-to-hard* generalization on MATH500 (Lightman et al., 2023). For math tasks, LLM-as-a-Judge utilizes Gemini 1.0 Pro, which we used for synthetic verification rationales for training. For each task, the generated solutions in Best-of-N are the same; the only difference is the verifier. Math tasks use model-generated verification rationales for training GenRM-CoT. Data will be released at: <https://sites.google.com/view/generative-reward-models>.

GenRM

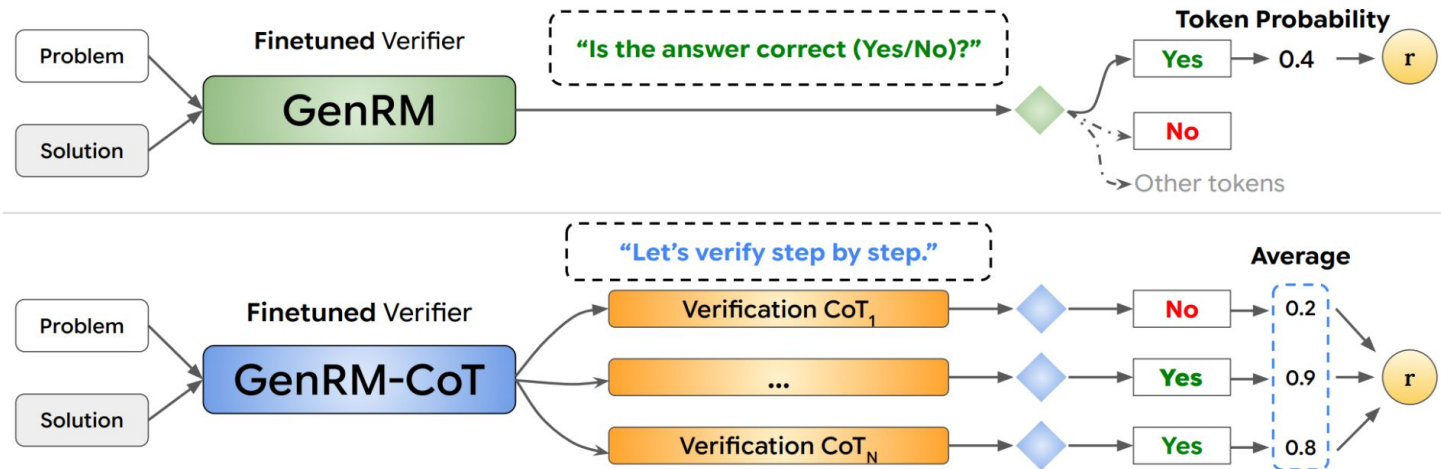


Figure 3 | **An illustration of generative verifiers**, namely GenRM and GenRM-CoT. Given a question and a candidate solution, GenRM directly finetunes an LLM to answer the question *'Is the answer correct (Yes/No)?'* via SFT on the next-token response corresponding to either 'Yes' or 'No'. During inference, the verifier score is obtained by extracting the probability of the 'Yes' token (4). In comparison, GenRM-CoT finetunes a LLM to produce verification chain-of-thought (CoT) rationale before yielding the final Yes/No token. At test-time, we sample multiple CoT rationales and use majority voting to compute the average probability of 'Yes', enabling GenRM-CoT to utilize additional inference-compute for better verification.

GenRM

Problem: Tim decides to cancel his cable subscription and get streaming services. He gets Netflix for \$10 a month. Hulu and Disney Plus normally cost \$10 a month each but he saves 20% for bundling. How much money does he save by cancelling his \$60 cable package?

Solution: Tim pays \$60 for cable. He gets Netflix for 10 and **the bundle of Hulu and Disney Plus costs $\$10 * 80\% = \8** . So he pays $\$10 + \$8 = \$18$ for the bundle. Now he saves $\$60 - \$18 = \$42$. The answer is 42.

Discriminative RM correctness score: **0.999**

GenRM-CoT: Let's verify step by step. ...

Step 3. ****The bundle of Hulu and Disney Plus costs $\$10 * 80\% = \8 ** (Incorrect)**

- Expected Answer: **The bundle of Hulu and Disney Plus costs $\$10 + \$10 = \$20$. With the 20% discount, the total cost is $\$20 * 0.8 = \16**

Verification: **Is the answer correct (Yes/No)? No**

GenRM-CoT (Majority Voting) score: **0.0015**

VersaPRM: Multi-Domain Process Reward Model via Synthetic Reasoning Data

Thomas Zeng¹ Shuibai Zhang¹ Shutong Wu¹ Christian Classen¹ Daewon Chae² Ethan Ewer¹ Minjae Lee³
Heeju Kim³ Wonjun Kang³ Jackson Kunde¹ Ying Fan¹ Jungtaek Kim¹ Hyung Il Koo³
Kannan Ramchandran⁴ Dimitris Papailiopoulos¹ Kangwook Lee¹

Abstract

Process Reward Models (PRMs) have proven effective at enhancing mathematical reasoning for Large Language Models (LLMs) by leveraging increased inference-time computation. However, they are predominantly trained on mathematical data and their generalizability to non-mathematical domains has not been rigorously studied. In response, this work first shows that current PRMs have poor performance in other domains. To address this limitation, we introduce *VersaPRM*, a multi-domain PRM trained on synthetic reasoning data generated using our novel data generation and annotation method. VersaPRM achieves consistent performance gains across diverse domains. For instance, in the MMLU-Pro category of Law, VersaPRM via weighted majority voting, achieves a 7.9% performance gain over the majority voting baseline—surpassing Qwen2.5-Math-PRM’s gain of 1.3%. We further contribute to the community by open-sourcing all data, code and models for VersaPRM.

1. Introduction

Large Language Models (LLMs) have demonstrated significant potential in tackling complex reasoning tasks. Specifically, they can employ a step-by-step Chain-of-Thought (CoT) approach to generate more accurate and reliable solutions (Wei et al., 2022b; Kojima et al., 2022; Yao et al., 2023; Madaan et al., 2023). Moreover, by using additional test-time computation, the reasoning performance of LLMs can be further enhanced (Snell et al., 2024; Yao et al., 2024).

An important and widely-adopted test-time computation method is using *external verifiers*, such as reward models

to rank multiple generated solutions and select the best answer (Lightman et al., 2024). Reward models evaluate the quality of solutions, helping guide LLMs toward better outputs. In particular, Outcome Reward Models (ORMs) are used to provide supervision based solely on the correctness of the final outcome. However, ORMs fail to address errors in intermediate steps, limiting their effectiveness for complex, multi-step reasoning tasks (Luo et al., 2024; Lightman et al., 2024; Sun et al., 2024). Because ORMs suffer from this limitation, Process Reward Models (PRMs) have been proposed to offer *fine-grained, step-by-step feedback* on the correctness of each reasoning step (Lightman et al., 2024; Uesato et al., 2022). PRMs have proven highly effective during inference, improving the reranking of generated solutions and guiding LLMs through search-based algorithms (Wan et al., 2024; Wang et al., 2024a).

Several studies have shown that PRMs trained on extensive process supervision significantly outperform ORMs in mathematical reasoning tasks, with notable improvements reported on datasets such as MATH500 and GSM800K (Luo et al., 2024; Lightman et al., 2024; Uesato et al., 2022). While substantial investigation has been made in creating training data (Lightman et al., 2024; Wang et al., 2024b), training PRMs (Xiong et al., 2024), and evaluation (Zheng et al., 2024) with respect to mathematical reasoning, the application of PRMs to *non-mathematical domains*—such as Biology, Chemistry, and Law—remains underexplored. To investigate the capability of math PRMs in non-mathematical domains, we test open-source math PRMs such as Math-Shepherd (Wang et al., 2024b) and Qwen-2.5-Math-PRM (Zheng et al., 2024). Not surprisingly, these PRMs demonstrate poor performance, indicating their limited domain generalizability. They exhibit only marginal improvements over the baseline in Law, Philosophy, and Biology as illustrated in Figure 1.

To address this limitation, we propose fine-tuning PRMs on a synthetically generated multi-domain CoT dataset, to significantly enhance reasoning capabilities beyond mathematics. We call this resulting multi-domain PRM *VersaPRM*, short for versatile PRM. Notably, by sampling questions from the MMLU-Pro dataset (Wang et al., 2024c), we generate CoTs to produce step-by-step reasoning using an

¹University of Wisconsin–Madison ²Korea University
³FuriosaAI ⁴University of California, Berkeley. Correspondence to: Kangwook Lee <kangwook.lee@wisc.edu>.

Proceedings of the 42nd International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025. Copyright 2025 by the author(s).

VersaPRM

- Current PRMs have poor performance in domains other than Math.
- Train a multi-domain PRM.

VersaPRM

Table 1. Results of two open-source math PRMs on different domains in MMLU-Pro-CoT-Eval (Unlabeled) when using WMV with min-aggregation on 16 CoTs generated per question using Llama-3.1-8B-Instruct. In parenthesis we report absolute difference between WMV and MV (WMV−MV). While WMV using math PRMs exhibits greater improvement in Math and Math-adjacent domains, there is no significant improvement on MV in other domains.

Category	MV	Math-Shepherd	Qwen-2.5-Math-PRM
All	57.15	57.66 (+0.51)	58.17 (+1.02)
All except math	56.61	57.01 (+0.40)	57.32 (+0.71)
Math	62.40	64.13 (+1.73)	67.20 (+4.80)
Chemistry	58.67	60.13 (+1.46)	60.67 (+2.00)
Physics	58.53	61.87 (+3.34)	61.47 (+2.94)
Biology	75.38	75.38 (+0.00)	75.69 (+0.31)
Psychology	61.60	61.47 (-0.13)	62.27 (+0.67)
Law	35.93	37.24 (+1.31)	36.28 (+0.35)
History	49.20	49.87 (+0.67)	49.40 (+0.20)
Philosophy	44.83	44.70 (-0.13)	45.17 (+0.34)

Data generation

CoT generation: Prompt Llama-3.1-8B-Instruct to produce step-by-step reasoning for each question.

Labeling: Ask Llama-3.1-70B-Instruct to evaluate each CoT using step-by-step reasoning and locate the earliest erroneous step. The followings are provided:

1. Question
2. Ground truth answer
3. Reference explanation
4. Step-by-step solution.

Steps are categorized as GOOD, OK, or BAD.

Data augmentation

Two synthetic augmentation methods

1. Counterfactual augmentation: generating additional examples of incorrect reasoning by prompt an LLM to modify steps in correct CoTs.
2. Prompting an LLM to rewrite CoT steps.

(Did not lead to significant performance improvements.)

Experiments

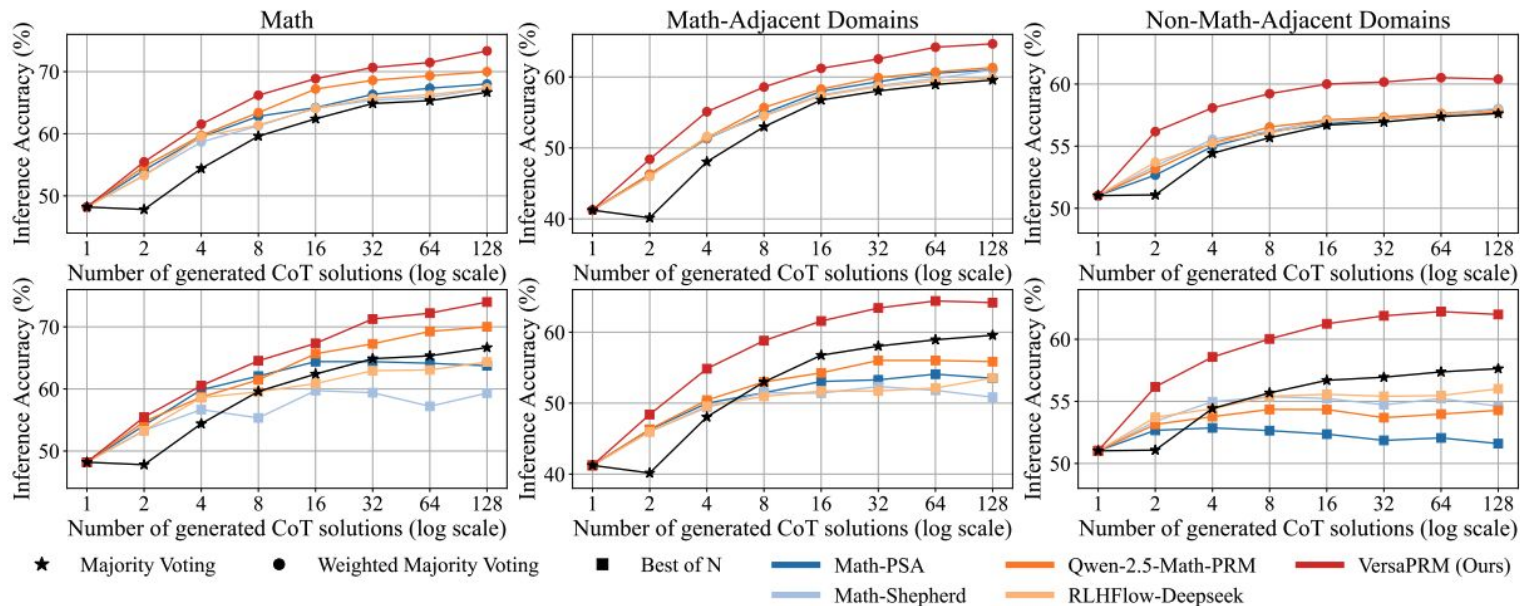


Figure 3. Comparison of WMV (top) and BoN (bottom) using VersaPRM against open-source math PRMs on MMLU-Pro-CoT-Eval (Unlabeled). We use min-aggregation and the CoTs are generated using Llama-3.1-8B-Instruct. VersaPRM has consistently better performance than math PRMs, and the differences become larger in domains not adjacent to Math.

Domain diversity of CoTs

Claim: Domain diversity of CoTs in a training dataset plays an integral role in generalization of PRMs to multiple domains.

Experiment 1: VersaPRM trained on Math-only subset vs random subset

Domain diversity of CoTs

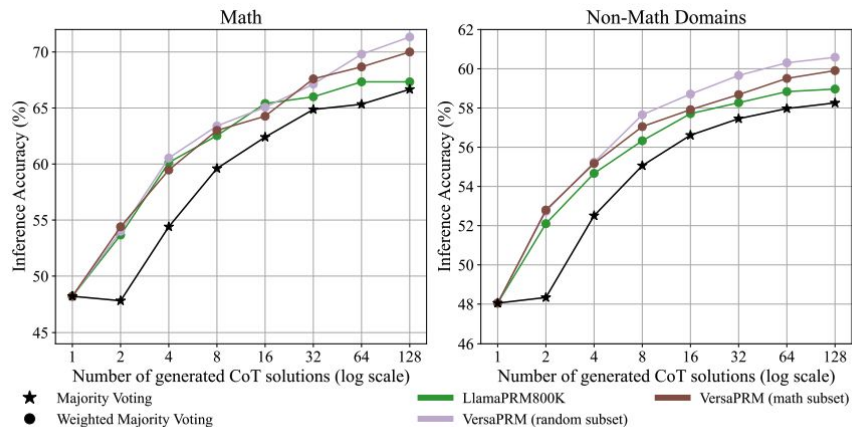


Figure 4. Comparison of WMV using LlamaPRM800K, VersaPRM (Math subset) and VersaPRM (random subset). VersaPRM (random subset) achieves better performance than VersaPRM (Math subset) in Math and non-Math.

Domain diversity of CoTs

Claim: Domain diversity of CoTs in a training dataset plays an integral role in generalization of PRMs to multiple domains.

Experiment 2: Heldout domain ablation experiment

Domain diversity of CoTs

E.7. Comparison of VersaPRM against VersaPRM Trained with One Categories Held-out

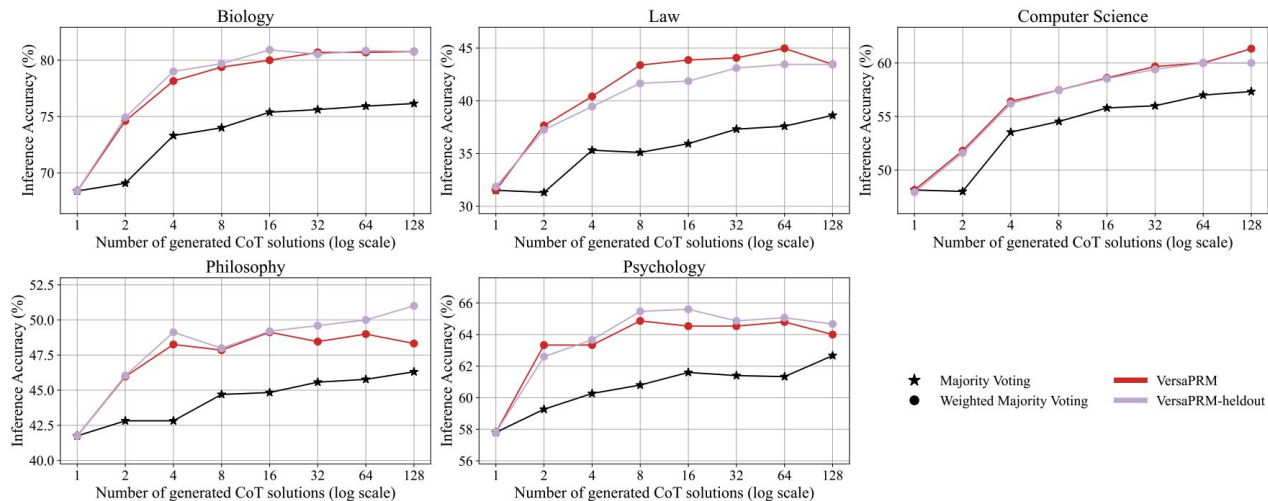


Figure 24. Hold-out domain ablation experiment results across all tested categories. WMV performance comparison of VersaPRM trained with each of the indicated domains held out versus the fully trained VersaPRM.

Other training configurations

Experiments Using Other Training Configurations.

While our final version of VersaPRM is trained from LlamaPRM800K on our synthetic data using LoRA, we also test the following training configurations on our multi-domain dataset:

- **VersaPRM (Llama Base):** We initialize training from Llama-3.1-8B-Instruct, and use LoRA fine-tuning with our multi-domain dataset.
- **VersaPRM (Qwen):** We initialize training from Qwen-PRM800K PRM, and utilize LoRA fine-tuning with our multi-domain dataset.
- **VersaPRM (full-tuned):** We initialize training from LlamaPRM800K PRM, and do *full* fine-tuning with our multi-domain dataset.

Other training configurations

Claim: Exposure to mathematical data beforehand can enhance a PRMs' ability to effectively leverage multi-domain CoT fine-tuning.

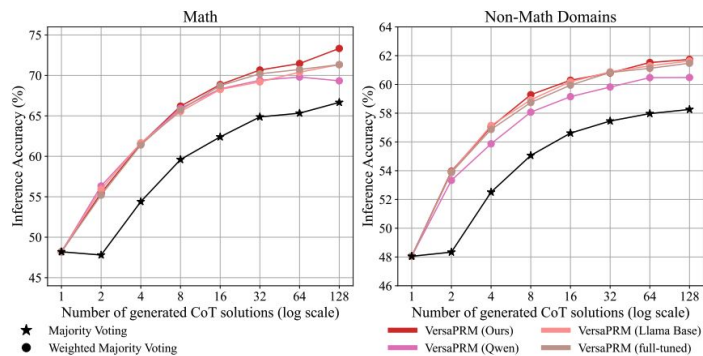
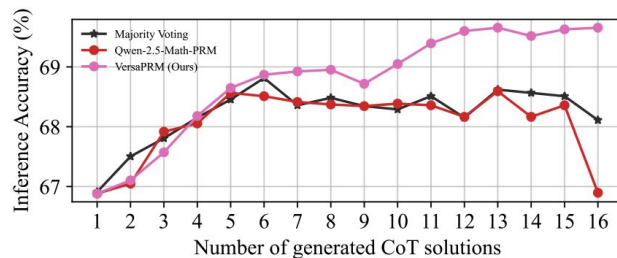
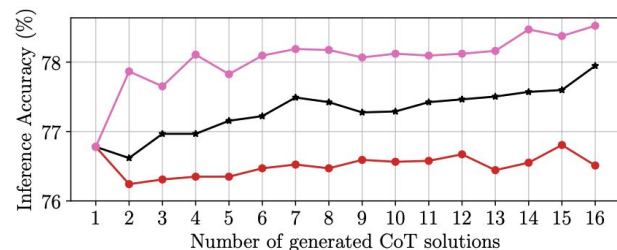


Figure 6. Comparison of MVW using VersaPRM against other multi-domain PRMs trained using different configurations. VersaPRM has better WMV performance than all other models in both Math and non-Math domains.

Does PRM with Test-Time Compute help Reasoning Models?



(a) Law Domain



(b) Philosophy Domain

Figure 8. Comparison of WMV using VersaPRM against Qwen-2.5-Math-PRM and Majority Voting on DeepSeek-R1 generated CoTs for Law and Philosophy subsets. VersaPRM consistently outperforms the math PRM and MV baseline in both subsets.

Reasoning-focused post-training

OpenAI o1 System Card

OpenAI

December 5, 2024

1 Introduction

The o1 model series is trained with large-scale reinforcement learning to reason using chain of thought. These advanced reasoning capabilities provide new avenues for improving the safety and robustness of our models. In particular, our models can reason about our safety policies in context when responding to potentially unsafe prompts, through deliberative alignment^[1]¹. This leads to state-of-the-art performance on certain benchmarks for risks such as generating illicit advice, choosing stereotyped responses, and succumbing to known jailbreaks. Training models to incorporate a chain of thought before answering has the potential to unlock substantial benefits, while also increasing potential risks that stem from heightened intelligence. Our results underscore the need for building robust alignment methods, extensively stress-testing their efficacy, and maintaining meticulous risk management protocols. This report outlines the safety work carried out for the OpenAI o1 and OpenAI o1-mini models, including safety evaluations, external red teaming, and Preparedness Framework evaluations.

2 Model data and training

The o1 large language model family is trained with reinforcement learning to perform complex reasoning. o1 thinks before it answers—it can produce a long chain of thought before responding to the user. OpenAI o1 is the next model in this series (previously OpenAI o1-preview), while OpenAI o1-mini is a faster version of this model that is particularly effective at coding. Through training, the models learn to refine their thinking process, try different strategies, and recognize their mistakes. Reasoning allows o1 models to follow specific guidelines and model policies we’ve set, helping them act in line with our safety expectations. This means they are better at providing helpful answers and resisting attempts to bypass safety rules, to avoid producing unsafe or inappropriate content.

The two models were pre-trained on diverse datasets, including a mix of publicly available data, proprietary data accessed through partnerships, and custom datasets developed in-house, which collectively contribute to the models’ robust reasoning and conversational capabilities.

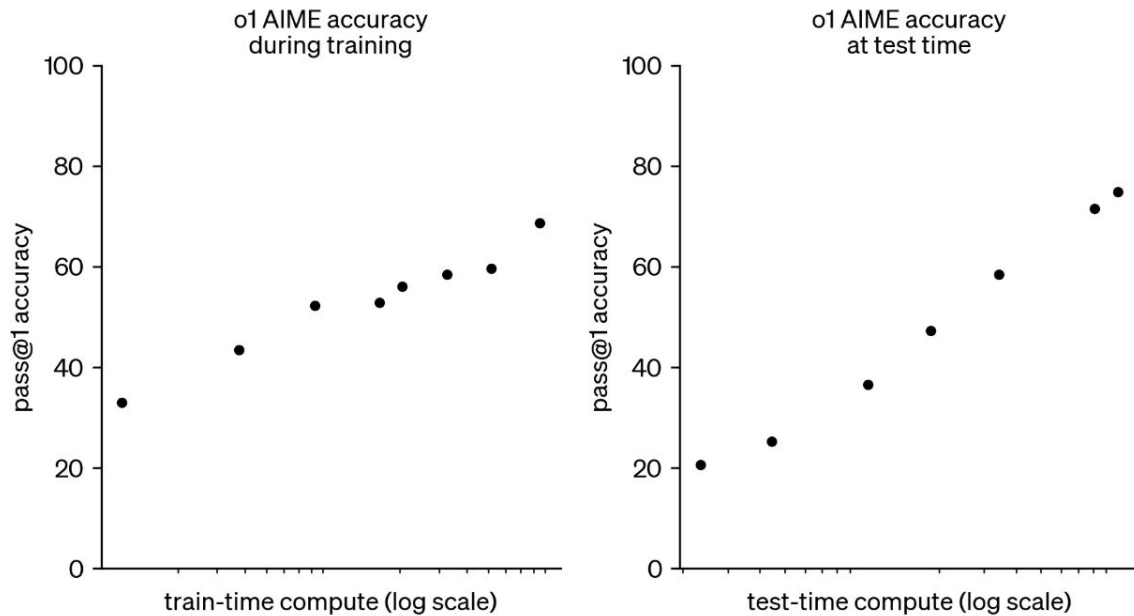
Select Public Data: Both models were trained on a variety of publicly available datasets, including web data and open-source datasets. Key components include reasoning data and

¹Deliberative alignment is a training approach that teaches LLMs to explicitly reason through safety specifications before producing an answer.

Chain-of-Thought

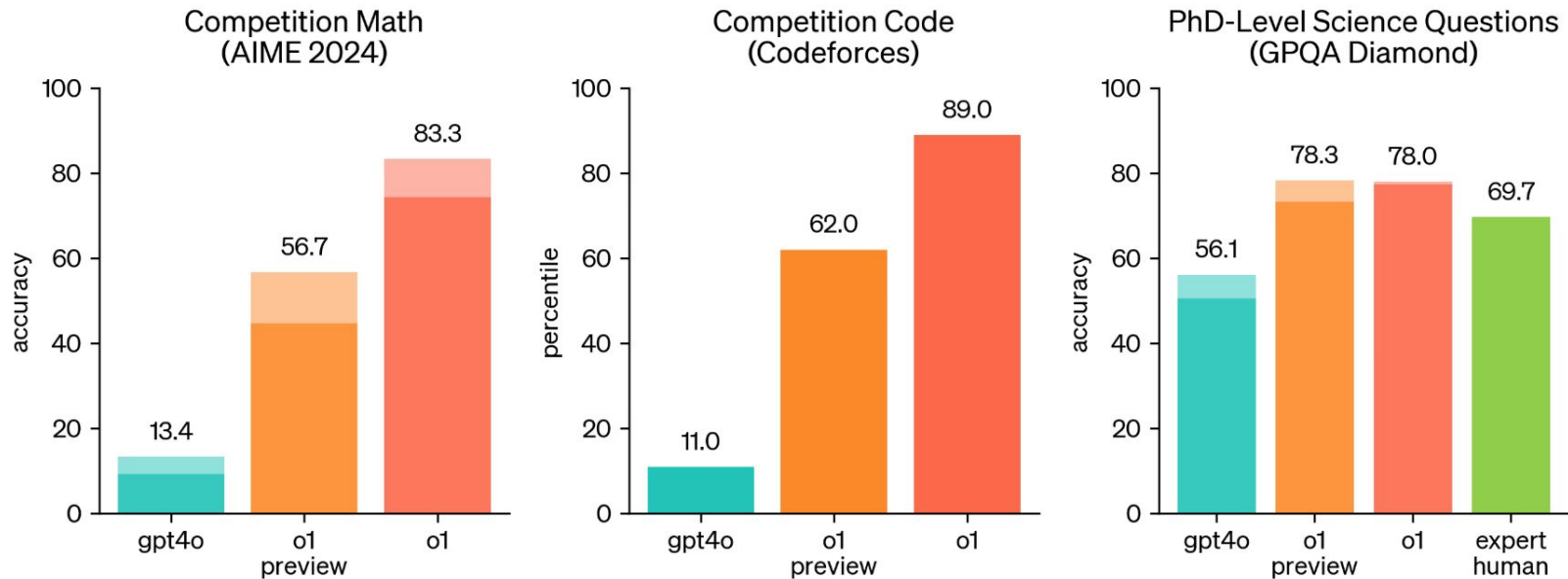
“Similar to how a human may think for a long time before responding to a difficult question, o1 uses a chain of thought when attempting to solve a problem. Through reinforcement learning, o1 learns to hone its chain of thought and refine the strategies it uses. It learns to recognize and correct its mistakes. It learns to break down tricky steps into simpler ones. It learns to try a different approach when the current one isn't working. This process dramatically improves the model's ability to reason.”

Test-time compute



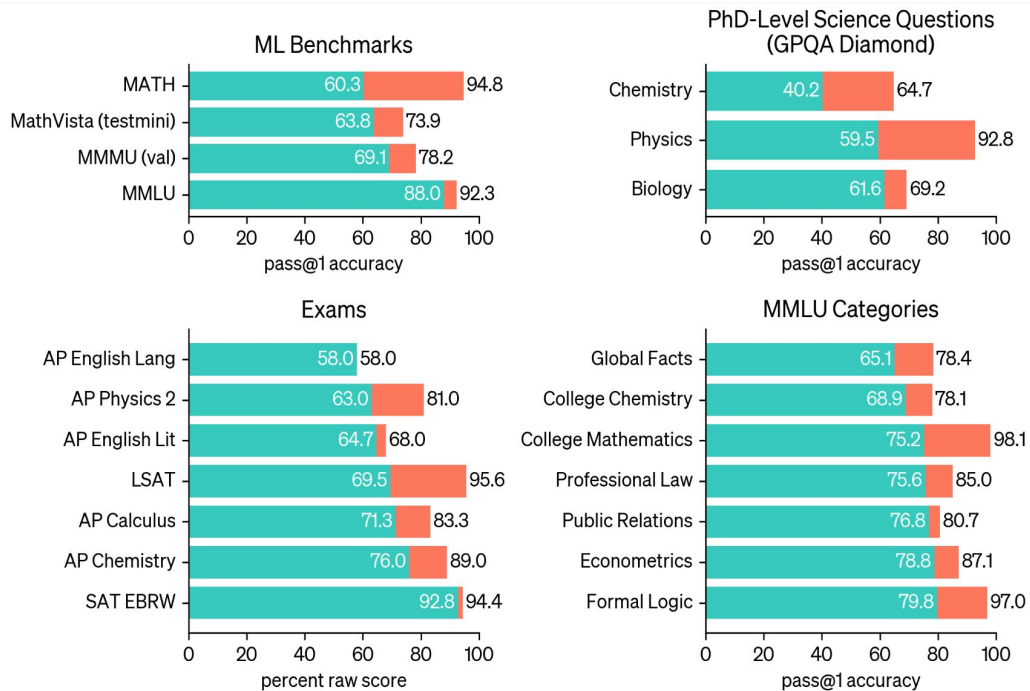
o1 performance smoothly improves with both train-time and test-time compute

Evals



o1 greatly improves over GPT-4o on challenging reasoning benchmarks. Solid bars show pass@1 accuracy and the shaded region shows the performance of majority vote (consensus) with 64 samples.

Evals



o1 improves over GPT-4o on a wide range of benchmarks, including 54/57 MMLU subcategories. Seven are shown for illustration.

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI

research@deepseek.com

Abstract

We introduce our first-generation reasoning models, DeepSeek-R1-Zero and DeepSeek-R1. DeepSeek-R1-Zero, a model trained via large-scale reinforcement learning (RL) without supervised fine-tuning (SFT) as a preliminary step, demonstrates remarkable reasoning capabilities. Through RL, DeepSeek-R1-Zero naturally emerges with numerous powerful and intriguing reasoning behaviors. However, it encounters challenges such as poor readability, and language mixing. To address these issues and further enhance reasoning performance, we introduce DeepSeek-R1, which incorporates multi-stage training and cold-start data before RL. DeepSeek-R1 achieves performance comparable to OpenAI-o1-1217 on reasoning tasks. To support the research community, we open-source DeepSeek-R1-Zero, DeepSeek-R1, and six dense models (1.5B, 7B, 8B, 14B, 32B, 70B) distilled from DeepSeek-R1 based on Qwen and Llama.

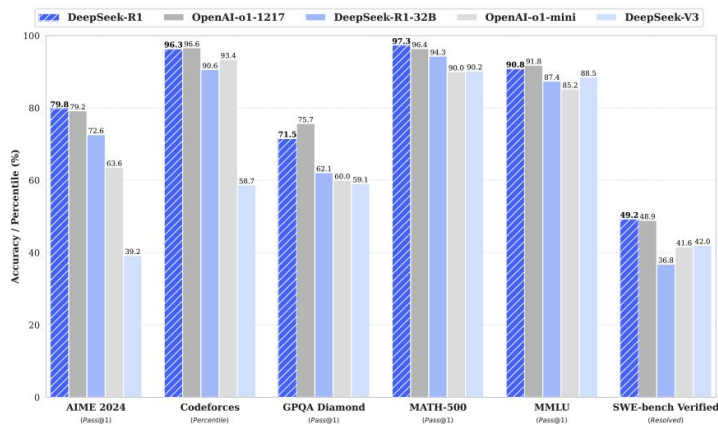


Figure 1 | Benchmark performance of DeepSeek-R1.

DeepSeek-R1-Zero

Start from base model without any supervised data.

GRPO

Group Relative Policy Optimization In order to save the training costs of RL, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which foregoes the critic model that is typically the same size as the policy model, and estimates the baseline from group scores instead. Specifically, for each question q , GRPO samples a group of outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_{θ} by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$
$$\frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \varepsilon, 1 + \varepsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) \right), \quad (1)$$

$$\mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - 1, \quad (2)$$

where ε and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \dots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}. \quad (3)$$

Reward Modeling

Accuracy reward: whether the response is correct

Format reward: if the model correctly place its thinking between `<think>` `</think>`

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within `<think>` `</think>` and `<answer>` `</answer>` tags, respectively, i.e., `<think>` reasoning process here `</think>` `<answer>` answer here `</answer>`. User: **prompt**. Assistant:

Table 1 | Template for DeepSeek-R1-Zero. **prompt** will be replaced with the specific reasoning question during training.

Performance

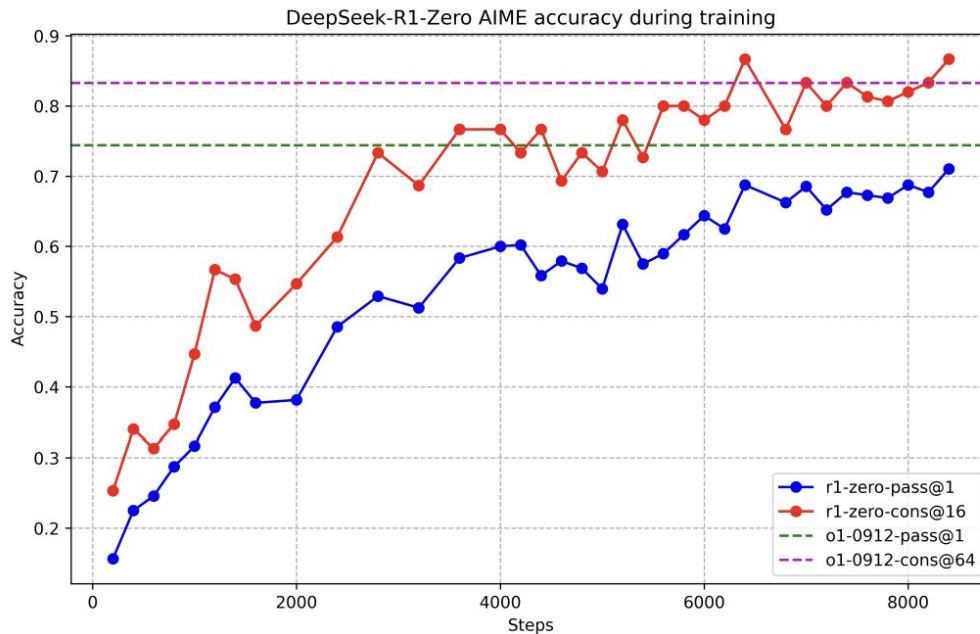


Figure 2 | AIME accuracy of DeepSeek-R1-Zero during training. For each question, we sample 16 responses and calculate the overall average accuracy to ensure a stable evaluation.

Performance

Model	AIME 2024		MATH-500	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
OpenAI-o1-mini	63.6	80.0	90.0	60.0	53.8	1820
OpenAI-o1-0912	74.4	83.3	94.8	77.3	63.4	1843
DeepSeek-R1-Zero	71.0	86.7	95.9	73.3	50.0	1444

Table 2 | Comparison of DeepSeek-R1-Zero and OpenAI o1 models on reasoning-related benchmarks.

Self-evolution Process of DeepSeek-R1-Zero

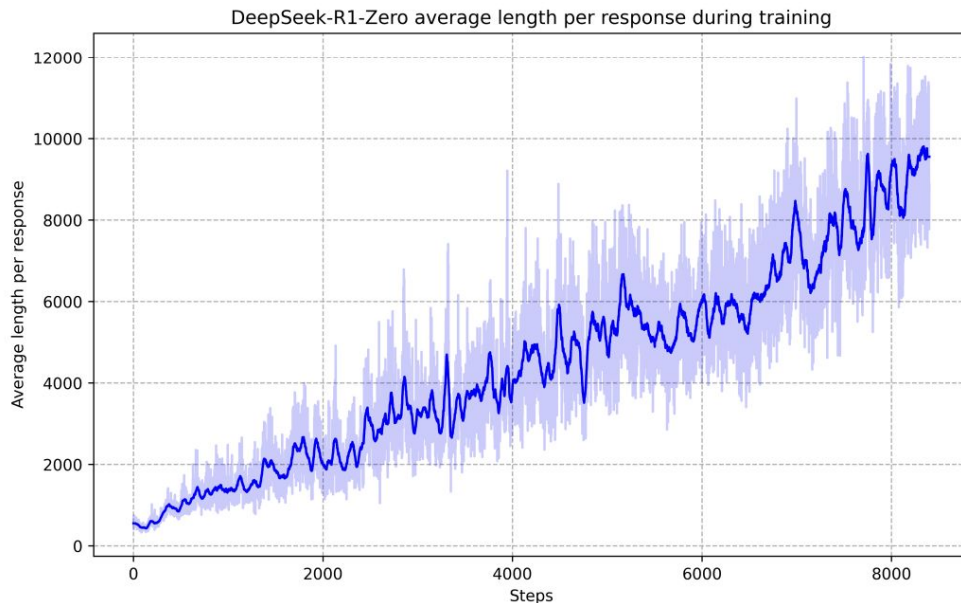


Figure 3 | The average response length of DeepSeek-R1-Zero on the training set during the RL process. DeepSeek-R1-Zero naturally learns to solve reasoning tasks with more thinking time.

“Behaviors such as reflection—where the model revisits and reevaluates its previous steps—and the exploration of alternative approaches to problem-solving arise spontaneously. **These behaviors are not explicitly programmed but instead emerge as a result of the model’s interaction with the reinforcement learning environment.** This spontaneous development significantly enhances DeepSeek-R1-Zero’s reasoning capabilities, enabling it to tackle more challenging tasks with greater efficiency and accuracy”

Aha Moment of DeepSeek-R1-Zero

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both ...

$$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2.$$

Rearrange to isolate the inner square root term:

$$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$$

...

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be ...

We started with the equation:

$$\sqrt{a - \sqrt{a + x}} = x$$

First, let's square both sides:

$$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$$

Next, I could square both sides again, treating the equation: ...

...

Table 3 | An interesting “aha moment” of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

“During this phase, DeepSeek-R1-Zero learns to allocate more thinking time to a problem by reevaluating its initial approach. This behavior is not only a testament to the model’s growing reasoning abilities but also a captivating example of how reinforcement learning can lead to unexpected and sophisticated outcomes.”

Drawback of DeepSeek-R1-Zero

- Poor readability
- Language mixing

DeepSeek-R1 Phase 1: SFT with code start data

Cold start data with several approaches

1. Few-shot prompting with long CoT as an example and prompt model to generate detailed answers with reflection and verification
2. Gathering DeepSeek-R1- Zero outputs in a readable format
3. Refining the results through post-processing by human annotators

DeepSeek-R1 Phase 2: RL

Employ RL as DeepSeek-R1-Zero in reasoning intensive tasks like coding, mathematics, science, and logic reasoning that have well-defined problems and clear solutions.

Added language consistency reward.

DeepSeek-R1 Phase 3: Rejection sampling and SFT

Use previous checkpoint to collect SFT data.

DeepSeek-R1 Phase 3: Rejection sampling and SFT

Reasoning data:

- Expand the dataset by incorporating additional data, some of which use a generative reward.
- Sample multiple responses and retain only the correct ones.
- 600K training samples.

DeepSeek-R1 Phase 3: Rejection sampling and SFT

Non-reasoning data (writing, factual QA, self-cognition, and translation):

- Adopt the DeepSeek-V3 pipeline and reuse portions of the SFT dataset of DeepSeek-V3.
- Call DeepSeek-V3 to generate a potential chain-of-thought before answering the question by prompting.
- 200K training samples

DeepSeek-R1 Phase 4: RL for all scenarios

Reasoning data: Rule-based reward, same as before.

General data: Use reward models to capture human preferences in complex and nuanced scenarios.

1. Helpfulness: If the summary emphasizes the utility and relevance of the response
2. Harmlessness: Evaluate the entire response of the model (CoT + answer)

Evaluation

Benchmark (Metric)		Claude-3.5- Sonnet-1022	GPT-4o 0513	DeepSeek V3	OpenAI o1-mini	OpenAI o1-1217	DeepSeek R1
Architecture		-	-	MoE	-	-	MoE
# Activated Params		-	-	37B	-	-	37B
# Total Params		-	-	671B	-	-	671B
English	MMLU (Pass@1)	88.3	87.2	88.5	85.2	91.8	90.8
	MMLU-Redux (EM)	88.9	88.0	89.1	86.7	-	92.9
	MMLU-Pro (EM)	78.0	72.6	75.9	80.3	-	84.0
	DROP (3-shot F1)	88.3	83.7	91.6	83.9	90.2	92.2
	IF-Eval (Prompt Strict)	86.5	84.3	86.1	84.8	-	83.3
	GPQA Diamond (Pass@1)	65.0	49.9	59.1	60.0	75.7	71.5
	SimpleQA (Correct)	28.4	38.2	24.9	7.0	47.0	30.1
	FRAMES (Acc.)	72.5	80.5	73.3	76.9	-	82.5
	AlpacaEval2.0 (LC-winrate)	52.0	51.1	70.0	57.8	-	87.6
	ArenaHard (GPT-4-1106)	85.2	80.4	85.5	92.0	-	92.3
Code	LiveCodeBench (Pass@1-COT)	38.9	32.9	36.2	53.8	63.4	65.9
	Codeforces (Percentile)	20.3	23.6	58.7	93.4	96.6	96.3
	Codeforces (Rating)	717	759	1134	1820	2061	2029
	SWE Verified (Resolved)	50.8	38.8	42.0	41.6	48.9	49.2
	Aider-Polyglot (Acc.)	45.3	16.0	49.6	32.9	61.7	53.3
Math	AIME 2024 (Pass@1)	16.0	9.3	39.2	63.6	79.2	79.8
	MATH-500 (Pass@1)	78.3	74.6	90.2	90.0	96.4	97.3
	CNMO 2024 (Pass@1)	13.1	10.8	43.2	67.6	-	78.8
Chinese	CLUEWSC (EM)	85.4	87.9	90.9	89.9	-	92.8
	C-Eval (EM)	76.7	76.0	86.5	68.9	-	91.8
	C-SimpleQA (Correct)	55.4	58.7	68.0	40.3	-	63.7

Table 4 | Comparison between DeepSeek-R1 and other representative models.